



PHP SESSION REDIRECT HTTP

2025

Silvio Priem Mendes, PhD

PHP State in WebApps

- State in webapps
 - Sessions
- HTTP redirect

PHP State in WebApps

- HTTP is a stateless protocol
 - which means that once a web server completes a client's request for a web page, the connection between the two goes away.
- In other words, there is no way for a server to recognize that a sequence of requests all originate from the same client.
- Particularly, in the universe of PHP programming, session management emerged as a direct response to the above mentioned problem, in cases where a Web application needs to keep track of its “state” during the occurrence of different HTTP requests.

PHP State in WebApps

- PHP offers 2 solutions for state maintenance:
 - Sessions (`$_SESSION` superglobal)
 - Data is kept on the server-side
 - Cookies (`$_COOKIES` superglobal)
 - Data is kept client-side

PHP Sessions

- A session is a way to store information (in variables) to be used across multiple pages.
- Session variables hold information about one single user, and are available to all pages in one application.
- A session is started with the **session_start()** function.
- A cookie called PHPSESSID is automatically sent to the user's computer to store unique session identification string.
- Session variables are set with the superglobal: **\$_SESSION**.

PHP Sessions

```
<?php
session_start();

if( isset( $_SESSION['counter'] ) )
{
    $_SESSION['counter'] += 1;
}
else
{
    $_SESSION['counter'] = 1;
}

$msg = "You have visited this page ". $_SESSION['counter'];
$msg .= "in this session.";

?>
<html>
<head>
<title>Setting up a PHP session</title>
</head>
<body>
<?php echo ( $msg ); ?>
</body>
</html>
```

PHP Sessions

- **Destroying a PHP Session**

- A PHP session can be destroyed by `session_destroy()` function. This function does not need any argument and a single call can destroy all the session variables.

```
<?php
    session_destroy();
?>
```

Example to unset a single variable:

```
<?php
    unset($_SESSION['counter']);
?>
```

PHP Sessions

1. Create a session (and a session ID) if one doesn't already exist:

```
session_start();
```

•2. If you are curious what the session ID looks like:

```
echo session_id();
```

3. Set a session variable that is stored on the web server and tied to the user's session ID:

```
$_SESSION["name"] = "Betty";
```

4. Get the value of a session variable that is tied to the user's session ID:

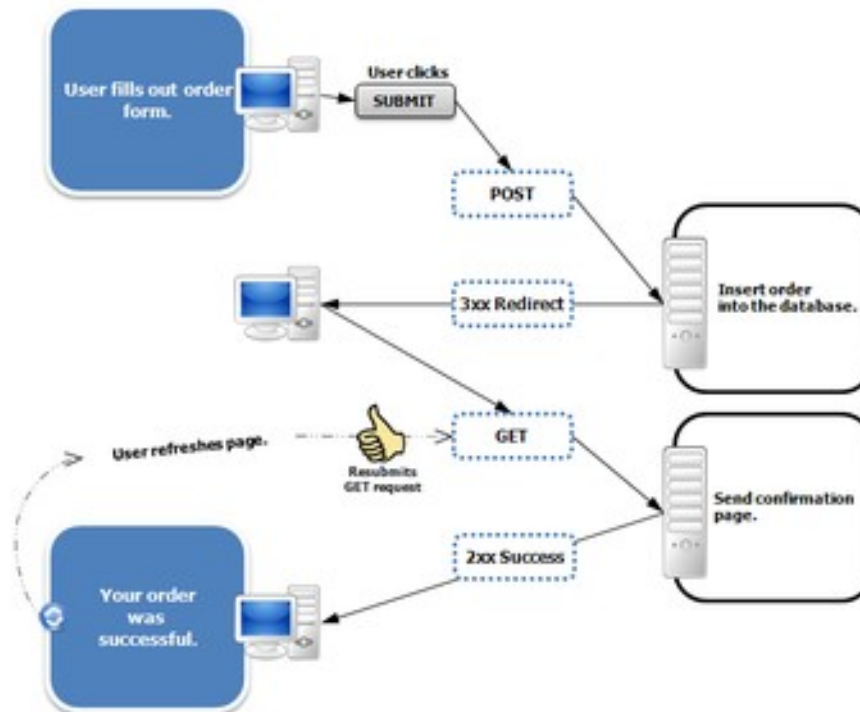
```
$auto = $_SESSION["name"];
```

5. Session variables are cleared when the browser is closed or when destroying the session explicitly:

```
session_destroy()
```


PHP HTTP Redirect

- HTTP Redirect



PHP HTTP Redirect

- `header()` is used to send a raw HTTP header.
 - See the HTTP/1.1 specification for more information on HTTP headers.
- Remember that `header()` must be called before any actual output is sent, either by normal HTML tags, blank lines in a file, or from PHP.
- It is a very common error to read code with `include`, or `require`, functions, or another file access function, and have spaces or empty lines that are output before `header()` is called. The same problem exists when using a single PHP/HTML file.

PHP HTTP Redirect

- Using header() to redirect

```
<html>
<?php

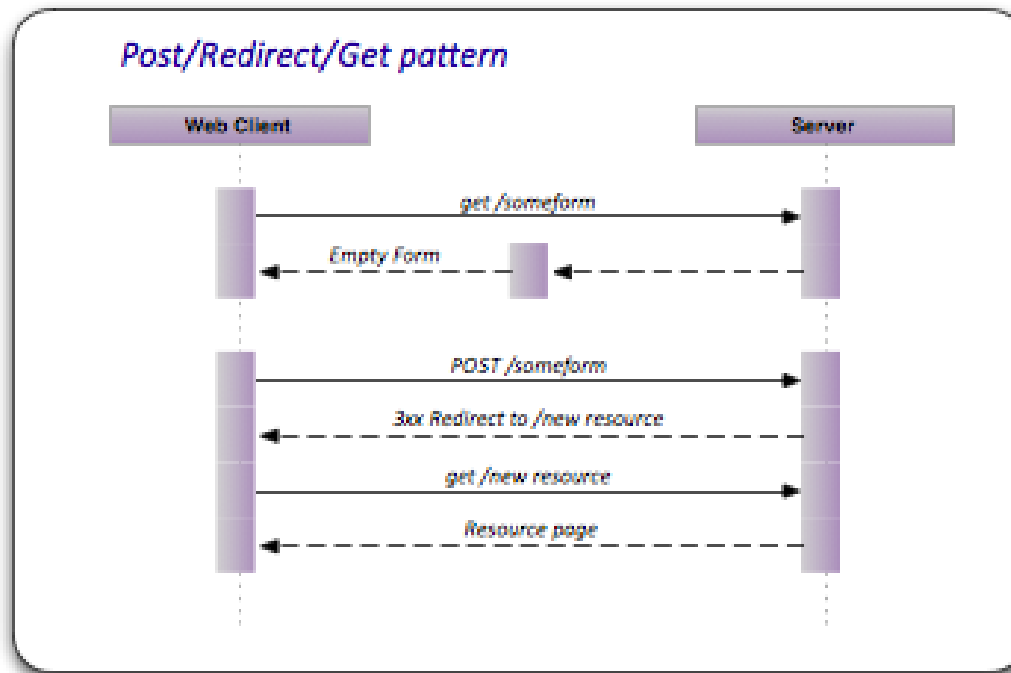
/* This will give an error. Note the output
 * above, which is before the header() call */

header('Location: http://www.example.com/');

?>
```

PHP HTTP Redirect

- How does it work?





BIBLIOGRAPHY